AL-NASSIM GOLDEN GROUP — Technical Stack & Project Handover Document

Project: nassim-platform (e-commerce storefront + admin/PIM + WhatsApp order notifications) **Project root:** `V:\14 SEPT WITH ONLY ORDER\5-8-26\backup 4-8-26\nassim-main` **Document date:** 10 November 2026 **Prepared from:** direct inspection of the source code, `package.json`, lockfiles, Prisma schema/migrations, middleware, configuration files and all API routes. Nothing in this document is assumed from prior discussions; where something could not be verified it is explicitly marked.

> **Privacy note:** this document intentionally contains **no** credentials, API keys, connection strings, phone numbers, WhatsApp session data or customer personal information. Sensitive values are shown as `<redacted>` placeholders. The `whatsapp-service/auth/` directory contains a **live WhatsApp session** and must never be copied, committed or shared.

Table of Contents

1. Technology Stack
2. Project Architecture
3. Project Folder Structure
4. Database Documentation
5. Main Features and Functional Modules
6. API Documentation
7. Environment Configuration
8. Installation and Local Setup (Windows)
9. Deployment and Infrastructure
10. Security and Maintenance
11. Known Limitations and Outstanding Work
12. Developer Quick Reference

---

1. Technology Stack

All versions below were read from `package-lock.json` (lockfileVersion 3) and verified against the installed `node_modules` where relevant.

1.1 Runtime & languages

Technology	Version (locked)	Purpose
Node.js	**v24.18.1** installed on the current dev machine. Next.js engine requirement: `^18.18.0 || ^19.8.0 || >= 20.0.0`. The WhatsApp service requires `>= 20.0.0`.	Runtime for Next.js and the WhatsApp service
npm	11.16.0 installed	Package manager (npm workspaces are **not** used; two separate `package-lock.json` files — root and `whatsapp-service/`)
TypeScript	5.9.3	All server code (`app/api`, `lib`, `middleware.ts`), the admin SPA (`app/admin/**.tsx`) and the WhatsApp service (`whatsapp-service/src`)
JavaScript (ES2022, vanilla)	—	All storefront pages (`public/*.html` + `public/assets/js/*.js`). tsconfig `allowJs: true`; storefront JS is plain browser JS with no framework or bundler
SQL (PostgreSQL)	—	Database (schema fully managed by Prisma migrations)

1.2 Frontend

Technology	Version	Where used / purpose
Static HTML + vanilla JS + Tailwind (compiled CSS)	—	The entire customer storefront: ~20 live pages in `public/` (e.g. `home5.html`, `product-view.html`, `cart.html`, `checkout5.html`, `order-send-modified4.html`, `order-detail.html`, `customer-order.html`, department/subcategory listing pages). There is **no React on the storefront**.
Tailwind CSS	3.4.19 (+ `@tailwindcss/forms` 0.5.9, `@tailwindcss/container-queries` 0.1.1, `autoprefixer` 10.5.4, PostCSS 8.5.26)	Two compiled stylesheets committed in `public/assets/css/`: `tailwind.css` (storefront, built by `npm run build:css` from `tailwind.config.js`) and `tailwind.admin.css` (admin, built by `npm run build:css:admin` from `tailwind.admin.config.js`). **Compiled CSS is committed — rebuild manually after class changes.** The Tailwind Play CDN was removed; CSS is self-hosted.
React / React DOM	19.2.7	Used **only** for the admin dashboard SPA (`app/admin/`), rendered through the Next.js App Router.
Custom i18n runtime (`public/assets/js/i18n.js` + `i18n-data.js`)	—	AR/EN storefront translations (generated from an approved xlsx via `translation-system/apply.cjs`). Text-level RTL (never flips `<html dir>`).
libphonenumber-js	1.13.15	Phone validation: vendored browser build at `public/assets/js/vendor/libphonenumber-mobile.js` (checkout), npm package server-side.
Self-hosted Google Fonts	—	`public/assets/fonts/` (immutable cache headers).
Web Push service worker	—	`public/sw.js` — **staff-only** new-order push notifications. No PWA manifest; no customer-facing offline caching.

State management: there is **no state-management library** anywhere. Storefront state lives in browser storage: `localStorage` keys `nassim_cart`, `nassim_wishlist`, `nassim-lang`, `color-theme`, `OWNER_KEY`; `sessionStorage` key `nassim_checkout_draft`. Admin SPA state is plain React `useState` inside one large client component (`app/admin/AdminApp.tsx`). Server cart/wishlist (`/api/cart`, `/api/wishlist`) is used when logged in and merged from guest storage at login.

1.3 Backend

Technology	Version	Where used / purpose
Next.js (App Router)	**15.5.24**	Serves: (a) all static storefront pages from `public/` (with rewrites/redirects/headers in `next.config.mjs`), (b) every REST API route under `app/api/**/route.ts` (Node runtime), (c) the admin SPA routes, (d) the `/o/[slug]` short-link route handler.
Route handlers (Web `Request` / `Response`)	—	49 API route files, all under `app/api/`.
Zod	3.25.76	Request-body validation in virtually every route.
Prisma Client	**6.19.3** (`@prisma/client`)	Sole database access point (`lib/db.ts`; single shared client). Preview feature `relationJoins` enabled.

1.4 Database

Technology	Version	Details
PostgreSQL	**18** (Neon, aws-us-east-2)	Hosted on **Neon** (serverless Postgres), project "al-nassim website", branch `production`, database `neondb`. See `DATABASE_SETUP.md`.
Prisma ORM	6.19.3 (client + CLI)	Schema at `prisma/schema.prisma` (PostgreSQL provider, 38 models / 8 enums); **18 migrations** in `prisma/migrations/`. `directUrl` (`DIRECT_URL` env) is used for migrations; runtime uses the pooled `DATABASE_URL`.
Neon CLI/config packages	`@neon/config` 1.9.3, `@neon/env` 1.6.4	`neon.ts` at project root declares branch policy (non-default branches auto-expire after 7 days).

1.5 Authentication & authorization

Technology	Purpose
`jsonwebtoken` 9.0.3	HS256 session JWT (`{uid, sid, role}`) carried in an HttpOnly cookie (`SESSION_COOKIE_NAME` , default `nassim_sid`), `SameSite=Strict` , `Secure` in production. **DB-backed sessions**: every request re-validates the `Session` row (`lib/auth.ts`).
`bcryptjs` 2.4.3	Password hashing, bcrypt cost **12** (`lib/security.ts`).
Edge middleware (`middleware.ts`)	Page gates for `/admin*` and `/internal-orders*` (Web Crypto HS256 verification, algorithm allow-list), login/register rate limiting (10 req/min/IP), Origin/Host check on all state-changing `/api/*` calls, security headers.
Roles	`CUSTOMER` , `STAFF` , `ADMIN` , `SUPERADMIN` (enum). Enforced gates: `requireStaff()` (ADMIN|STAFF) and `requireAdmin()` (ADMIN only). **SUPERADMIN is no longer admitted anywhere** (per comment in `lib/auth.ts`). STAFF/ADMIN split is enforced server-side in every admin route; the admin UI mirrors it.

1.6 Images, files & storage

Technology	Purpose
`sharp` 0.35.4	Admin image upload pipeline (`app/api/admin/upload/route.ts`): magic-byte sniffing, 8 MB/file cap, EXIF auto-rotate, resize to ≤1600px (env-tunable), WebP conversion (quality 82 default). **No cloud storage — files are written to local disk at `public/uploads/`** (52 files, ~35 MB currently).
Local disk `public/uploads/`	The only image/file storage. Served statically at `/uploads/...` . Product URLs stored in `Product.image` / `Product.images` (string / string array).
lh3.googleusercontent.com	Allowed in CSP `img-src` for product images imported historically from a Google Shopping feed. No live import feature exists.

1.7 WhatsApp integration (Baileys)

Technology	Version (locked)	Purpose
Baileys	**7.0.0-rc14**	Unofficial WhatsApp Web protocol library. Powers a **standalone Express service** in `whatsapp-service/` (separate npm project, Node ≥ 20).
Express	4.22.3 (locked; `^4.21.2` declared)	HTTP API of the WhatsApp service on `127.0.0.1:4000` (dev, forced by the service manager).
pino / qrcode / dotenv	9.14.0 / 1.5.4 / 16.6.1	Logging / QR data-URL generation / service env loading.
TypeScript (service)	5.9.3	`whatsapp-service/src/*.ts` compiled to `whatsapp-service/dist/` (`npm run build = tsc`).

Architecture details in §2.9 and §5.4.

1.8 Other third-party services

Technology	Version	Purpose
`resend`	6.18.0	Transactional email — **password-reset emails only** (`lib/email.ts`).
`web-push`	3.6.7	Staff new-order Web Push (VAPID). Subscriptions stored as JSON rows in the `Setting` table (`lib/web-push.ts`).
Neon Postgres	—	See §1.4.

1.9 Dev/build tools & notable packages

Tool	Version	Notes
`prisma` CLI	6.19.3	`prisma generate / migrate dev / migrate deploy` ; seed = `node prisma/seed.mjs` .
`typescript + tsc --noEmit`	5.9.3	`npm run typecheck` .

Tool	Version	Notes
Tailwind CLI	3.4.19	`npm run build:css`, `npm run build:css:admin`.
`exceljs` **4.4.0**	Declared but **never imported anywhere** in `app/`, `lib/`, `scripts/`, `prisma/` or root scripts — dead dependency (see §11).	
`recharts` **3.10.1**	Declared but **never imported** — the admin dashboard uses plain HTML cards/lists, not charts. Dead dependency.	
`jsdom` **24.1.3**	Declared but **never imported** anywhere in the project's own code. Dead dependency.	
mkcert	Optional	`npm run dev:https:certs` generates LAN dev certificates into `certs/` (script hardcodes LAN IP `192.168.1.172`). `certs/` is git-ignored.
No test framework, no linter config, no ESLint, no CI	—	`npm run typecheck` is the only automated check.

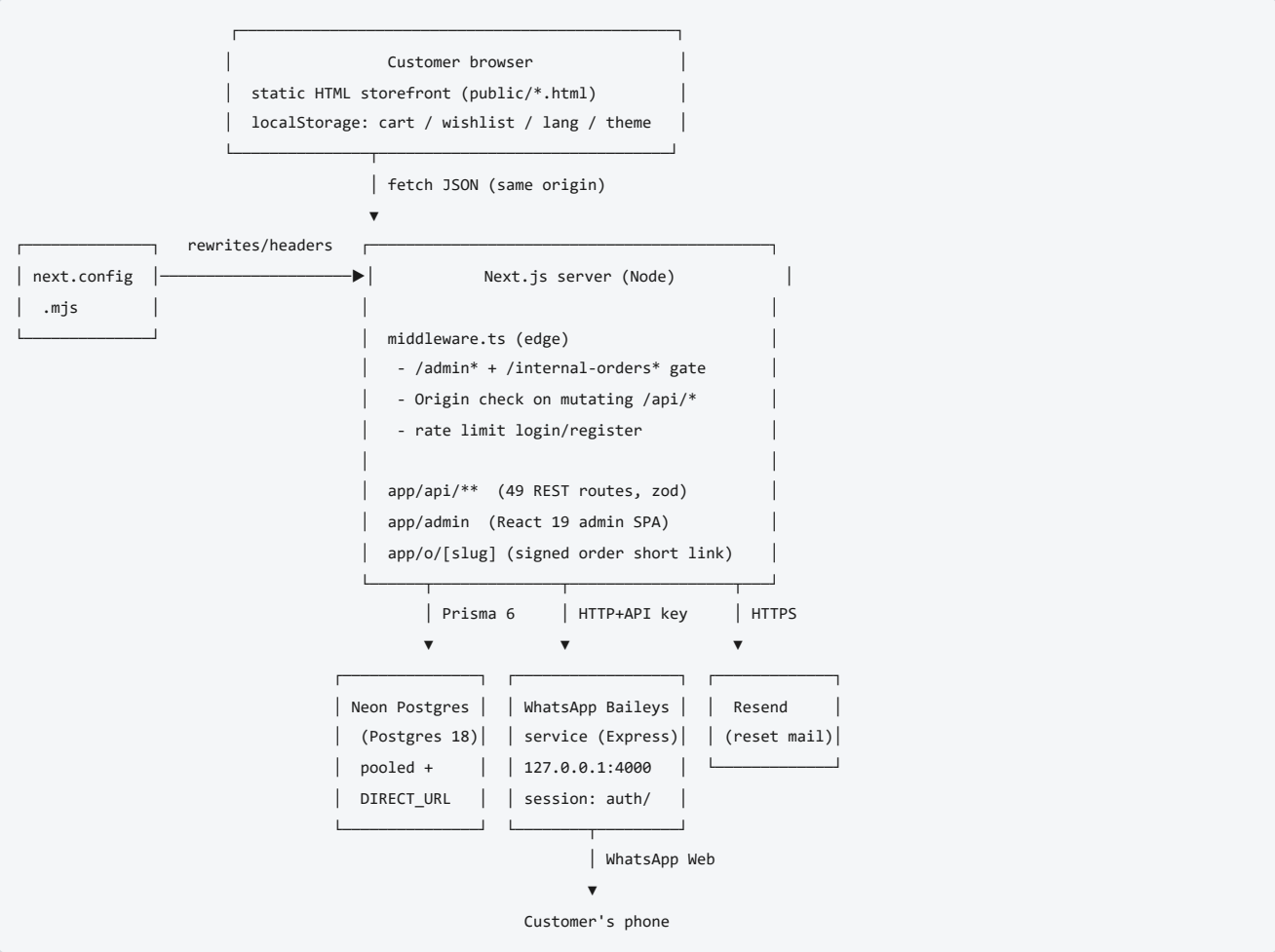
---

2. Project Architecture

2.1 Big picture

The system is **three cooperating pieces**:

1. **Customer storefront** — static HTML pages + vanilla JS in `public/`, served by Next.js (with `/` rewritten to `home5.html`). All dynamic data comes from the project's own JSON/JS APIs under `/api/*`. No server-side rendering of storefront pages.
2. **Next.js application server** — App Router: 49 REST API routes (Prisma/PostgreSQL), the React admin SPA (`/admin`), the staff order pages (`/internal-orders.html` legacy and `/internal-orders2.html` primary), the signed order short-link (`/o/[slug]`), edge middleware (auth gates, CSRF-origin check, rate limits, headers), plus `next.config.mjs` rewrites/redirects/cache-headers/CSP.
3. **WhatsApp Baileys service** — a separate Node/Express/TypeScript process (`whatsapp-service/`) bound to `127.0.0.1:4000`, holding one WhatsApp Web session. The Next.js server is its only client (server-to-server, API-key authenticated).



Staff browsers load the auth-gated staff pages (`internal-orders2.html` primary; `internal-orders.html` legacy) and subscribe to Web Push via `public/sw.js` ; push messages are sent by the Next server using `web-push` when a new order arrives.

2.2 Frontend structure

- **Storefront:** each page is a self-contained HTML file that loads compiled Tailwind CSS, shared JS from `public/assets/js/` (i18n, cart, checkout, search, nav, filters, account) and inline page scripts. Product data is fetched once from **`GET /api/catalog`** , which returns JavaScript defining `window.NASSIM_PRODUCTS` (10 s max-age, in-process cache in `lib/catalog-cache.ts`). Filtering, sorting and pagination are all **client-side**.
- **Admin SPA:** every `/admin/*` route renders the same client component `app/admin/AdminApp.tsx` (≈1,100 lines) with `initialTab` ; tabs: Dashboard, Products, Orders, Master Data, Customers, Staff, Audit Log, Backup & Restore, Settings. Supporting components in `app/admin/_components/` (largest: `ProductsTab.tsx` ≈1,340 lines). Data access is raw `fetch` with `credentials: "same-origin"` .
- **Staff order pages (both edge-gated, redirect to `/admin/login` when unauthenticated):**
  - `public/internal-orders2.html` — the **primary, automation-enabled staff page**: order document view with print, per-item verification, packing, status updates, the "LINK WHATSAPP" connection panel (`/api/whatsapp/config|qr|status|disconnect`), manual WhatsApp send (`/api/whatsapp/send`), push subscription and sound alerts. **STAFF-role logins land here by default** (`app/admin/login/page.tsx` , `STAFF_LOGIN_DESTINATION`).
  - `public/internal-orders.html` — the **legacy** staff order page (order list, wa.me manual links, print, push subscription); still reachable directly and still the Web-Push deep-link target (`lib/web-push.ts` links `/internal-orders.html?order=…`).

2.3 Backend/API structure

- All endpoints are Next.js route handlers (`app/api/**/route.ts`) with zod-validated bodies and the response envelope `{ success, data, error }` (some legacy routes also expose duplicate keys for backward compatibility).
- Shared logic lives in `lib/` : auth (`auth.ts`), security primitives (`security.ts`), order workflow state machine (`order-workflow.ts`), order access/tokens (`order-access.ts` , `order-link.ts`), stock ledger (`online-stock.ts`), shipping rules (`shipping.ts`), WhatsApp automation (`whatsapp-automation.ts`), web push (`web-push.ts`), email (`email.ts`), catalog cache, backup ZIP writer (`backup-zip.ts`), custom fields, product variants, report columns, low stock, analytics.

- **No queue, no cron, no background workers** — post-checkout notifications are fire-and-forget `void` promises inside the request process.

2.4 Database communication

- Single Prisma client singleton (`lib/db.ts`); importing `lib/env.ts` validates required env vars at startup and fails loudly.
- Runtime uses the Neon **pooled** connection string; migrations use `DIRECT_URL` (direct endpoint, `sslmode=require`).
- The schema's legacy PIM tables (Department/Section/Category/… and `Order`) are owned by the migration role; the least-privilege app role cannot alter them, so several companion tables use **natural-id keys without foreign keys** (deliberate, documented in schema comments): `OrderSecurity` , `AttributeI18n` , `AttributeOptionI18n` , `ProductCustomValueI18n` , `ColorI18n` , `MaterialI18n` , `CustomAttribute.categoryId` .

2.5 Authentication flow

1. `POST /api/auth/login` verifies bcrypt password → `createSession()` inserts a `Session` row (30-day expiry) and sets the HS256 JWT cookie `{uid, sid, role}` .
2. Every request: `getSessionUser()` verifies the JWT **and** re-reads the `Session` row (expired session or deactivated user ⇒ unauthenticated).
3. Edge middleware gates the `/admin*` and `/internal-orders*` *pages* using the JWT role claim (Web Crypto HS256; unsigned `jwt.decode` fallback only if `JWT_SECRET` is absent from the edge runtime — dev case). API routes always do the authoritative DB-backed check.
4. Guests can act without an account: cart/wishlist in localStorage, checkout via `POST /api/orders` (rate-limited, Origin-checked, idempotency-keyed), and order tracking via a one-time **capability token** (raw token shown once at checkout; only its SHA-256 hash is stored in `OrderSecurity`) or the signed `/o/{id}-{hmac16}` short link.

2.6 Product catalogue & PIM architecture

- Taxonomy: **Department** → **Section (optional)** → **Category** → **Subcategory**; products attach to a Category and optionally a Subcategory. Master data: Brand, Material, Color (+AR), Size, Supplier, Unit, Country (Tax retained but its admin CRUD was deliberately removed).
- **Dynamic custom attributes** (`CustomAttribute`) are defined per Subcategory (or per Category fallback) with section placement (HEADER/DESCRIPTION/SPECIFICATIONS), field types (TEXT/LONG_TEXT/NUMBER/SELECT ± multiSelect) and Arabic i18n side tables; per-product values in `ProductCustomValue` (+ `ProductCustomValueI18n` for Arabic).
- **Variants/related grouping** uses the internal `classCode` string: products sharing a class code form a variant/related group; the public product API derives customer-safe `variantOptions` from siblings and never exposes `classCode` .
- Admin product management (ADMIN-only API): create/update, soft delete (`isActive=false` — there is no hard delete), publish state, custom values, related products (`RelatedProduct`), image upload (sharp→WebP→ `public/uploads`), stock changes (audited), low-stock view, printable product report (client-rendered; server returns JSON only).
- **Versioning/history** = the `AuditLog` (e.g. `PRODUCT_UPDATE` , `PRODUCT_STOCK_CHANGE` old→new) — there is no version table or rollback. **Duplicate detection** = slug uniqueness + duplicate pre-checks in the product form. **Bulk edit / Excel import** do not exist in the running app (`exceljs` unused).

2.7 Customer order lifecycle

1. **Checkout** (`checkout5.html` + `assets/js/checkout.js`): customer details (phone validated with vendored libphonenumber metadata, country/governorate/area from `nassim-locations-data.js` — 6 governorates / 134 areas), shipping fee from `GET /api/shipping` (display only), draft autosave in sessionStorage.
2. **Submission** `POST /api/orders` : rate limit 8 orders / 10 min / IP; zod validation (1–100 items); **prices always recomputed from the DB** (client prices ignored; `shipping` is not accepted from the client); optional `x-idempotency-key` header (replay returns the original order without re-issuing the tracking token); **no stock deduction at placement** (a website order is a *request*; availability is confirmed by staff); order number from Postgres sequence `order_number_seq` (`ORD-<n>`); permanent ownership snapshot (`originalUserId` / `originalUserEmail`) survives account deletion; server cart cleaned for logged-in users.
3. **Tracking token**: 32-char random token returned **once**; SHA-256 hash stored in `OrderSecurity` (1:1 with order via globally-unique `orderNumber` , no FK).
4. **Post-commit notifications**: staff Web Push (deep link to `/internal-orders.html?order=…`) + fire-and-forget automatic WhatsApp to the customer (gated by the automation Setting; TEST MODE redirects to allow-listed numbers).
5. **Status machine** (enforced by `lib/order-workflow.ts` ; every transition is re-validated under a `SELECT … FOR UPDATE` row lock; illegal or raced moves return HTTP 409):

From	Allowed next status	Performed by
PENDING	SEEN, UNDER_REVIEW, CANCELLED_BY_STAFF	staff (SEEN = staff opened the order sheet)
SEEN	UNDER_REVIEW, CANCELLED_BY_STAFF	staff
UNDER_REVIEW	READY_FOR_CONFIRMATION, CANCELLED_BY_STAFF	staff
READY_FOR_CONFIRMATION	CONFIRMED, CANCELLED_BY_CUSTOMER, CANCELLED_BY_STAFF	**customer** (confirm/cancel) or staff
CONFIRMED	PACKING, CANCELLED_BY_STAFF	staff
PACKING	READY_FOR_DELIVERY, CANCELLED_BY_STAFF	staff
READY_FOR_DELIVERY	OUT_FOR_DELIVERY	staff
OUT_FOR_DELIVERY	DELIVERED	staff
DELIVERED	COMPLETED	staff
COMPLETED, CANCELLED_BY_CUSTOMER, CANCELLED_BY_STAFF	— (terminal)	—

- Customer may **confirm** (READY_FOR_CONFIRMATION → CONFIRMED; items marked `UNAVAILABLE` become `REMOVED_AFTER_CONFIRMATION` and drop out of the billable total) or **cancel** (→ CANCELLED_BY_CUSTOMER) via `POST /api/orders/[id]/confirm` (owner session or guest token).
- **Stock note (verified):** `lib/online-stock.ts` defines `reserveOnlineStock()`, but **nothing in the codebase calls it** — stock is never deducted at order placement. The cancel/confirm/decline paths do call `restoreOnlineStockForOrder()` / `restoreStockForRemovedItems()`, but these are ledger-capped, so for orders with no reservation they are safe no-ops. The header comment of `lib/online-stock.ts` still describes reservation-at-placement and is stale. `Product.stock` therefore changes only through admin edits.
- Staff perform verify (per-item AVAILABLE/UNAVAILABLE, auto PENDING→UNDER_REVIEW), status moves, pack (`pack|ready|deliver|complete` + optional POS status), timeline, and the signed order link.

2.8 Staff order management

- `public/internal-orders2.html` (primary; where STAFF logins land) and `public/internal-orders.html` (legacy): polling order list (`/api/admin/orders`), status changes, item verification, packing, printing, notification sound, Web Push subscription. The WhatsApp connection panel and automated manual-send button exist **only on** `internal-orders2.html`; the legacy page offers pre-filled `wa.me` links instead. Server-side everything is `requireStaff()`; ADMIN-only actions are additionally gated per route.

2.9 WhatsApp notification architecture (summary; details §5.4)

- **Automatic path:** order created → `void autoNotifyAfterOrder(order.id)` → gates (automation Setting ON, TEST MODE recipient resolution) → atomic claim on `Order.whatsappStatus` (PENDING/SENDING/SENT/FAILED with 2-minute stale-claim expiry) → `POST 127.0.0.1:4000/send-message` with idempotency key `order-notif-<orderId>` → success stamps `whatsappSent/SENT/Auto/SentAt` + audit row; failure stamps FAILED (staff can retry manually).
- **Manual path:** staff button → `POST /api/whatsapp/send {orderId}` → same `notifyOrder()` with `source:"manual"`.
- **Connection management:** staff "LINK WHATSAPP" panel in `internal-orders2.html` → Next proxies to the service's `/qr`, `/status`, `/logout`.

2.10 Image handling & storage

- Upload: ADMIN-only `POST /api/admin/upload` → magic-byte sniffed (JPEG/PNG/WebP/GIF/AVIF, ≤8 MB) → sharp auto-rotate + resize (≤1600px) + WebP q82 → `public/uploads/{timestamp}-{rand}.webp` → URL `/uploads/...` attached to the product by a subsequent product PATCH/POST. No thumbnails/multi-size derivatives. No cloud storage.
- Product images: `Product.image` (primary string URL) + `Product.images[]`; the ordered `ProductImage` (`mediaImages`) table is written only by the restore tooling.

2.11 Admin dashboard architecture

See §2.2 and §6 (category Admin). Role split: STAFF sees Dashboard/Orders/Customers(read-only); ADMIN sees everything; there is no SUPERADMIN UI. All enforcement is server-side in the API routes.

---

3. Project Folder Structure

```
nassim-main/
├── app/                          # Next.js App Router (server code + admin SPA)
│   ├── layout.tsx                # minimal root layout (title/favicon only)
│   ├── admin/                    # React admin dashboard
│   │   ├── AdminApp.tsx          # the whole SPA (~1,100 lines): tabs, routing by tab
│   │   ├── _components/          # ProductsTab, CustomFieldsSection, MasterDataPanel,
│   │   │                         #   StaffTab, CustomersTab, SettingsTab (WhatsApp
│   │   │                         #   automation), BackupTab, ReportDialog, SpecsEditor,
│   │   │                         #   RelatedProductsPicker, HierarchyTree, ConfirmDialog
│   │   ├── login/  customers/  master-data/  orders/  products/
│   │   ├── print-report/         # printable product report (browser print → PDF)
│   │   └── page.tsx              # thin wrappers rendering <AdminApp initialTab=...>
│   ├── o/[slug]/route.ts         # signed customer order short link (GET handler)
│   └── api/                      # ALL REST endpoints (see §6)
│       ├── auth/        login, logout, me, register, delete-account, password-reset/{request,reset}
│       ├── cart/  wishlist/  addresses/  notifications/
│       ├── products/  products/[slug]/  catalog/  categories/  shipping/
│       ├── orders/  orders/[id]/  orders/[id]/confirm/
│       ├── staff/push/           # staff Web Push subscription management
│       ├── whatsapp/             # config, qr, status, send, disconnect (Next→service proxy)
│       └── admin/                # products, orders(+verify/pack/timeline/order-link),
│                                 #   customers, staff, hierarchy, master, attributes,
│                                 #   upload, stats, low-stock, audit-log,
│                                 #   backup/{products,orders}, restore/{products,orders},
│                                 #   whatsapp-automation
├── lib/                          # shared server logic (all TypeScript)
│   ├── auth.ts                   # session create/verify/destroy, requireStaff/requireAdmin
│   ├── security.ts               # bcrypt(cost 12), rate limit, CSRF helpers (unused), client IP
│   ├── db.ts / env.ts            # Prisma singleton + startup env validation
│   ├── order-workflow.ts         # order/item status state machine + locked transitions
│   ├── order-access.ts           # guest capability tokens (SHA-256 hashed)
│   ├── order-link.ts             # signed /o/{id}-{hmac16} short links
│   ├── online-stock.ts           # online stock ledger (reserve/restore)
│   ├── shipping.ts               # shipping rules from Settings
│   ├── whatsapp-automation.ts    # auto/manual notification pipeline, TEST MODE
│   ├── web-push.ts / email.ts    # VAPID push to staff / Resend reset emails
│   ├── backup-zip.ts             # dependency-free ZIP writer for backups
│   ├── catalog-cache.ts / custom-fields.ts / product-variants.ts
│   └── report-columns.ts / low-stock.ts / analytics.ts
├── prisma/
│   ├── schema.prisma             # 40+ models/enums (see §4)
│   ├── migrations/               # 18 migrations, 2026-07-20 → 2026-11-09
│   └── seed.mjs                  # seeds admin/owner/staff accounts; products only if
│                                 #   public/assets/js/products.js exists (removed by default)
├── public/                       # THE CUSTOMER STOREFRONT + static assets
│   ├── home5.html                # live homepage (rewritten from "/")
│   ├── product-view.html  cart.html  checkout5.html  order-send-modified4.html
│   ├── order-detail.html  customer-order.html  wishlist.html  password-reset.html
│   ├── internal-orders2.html     # PRIMARY staff page: orders, print, WhatsApp panel (auth-gated)
│   ├── internal-orders.html      # legacy staff order page (wa.me links; push deep-link target)
│   ├── houseware-*.html supermarket*.html warehouse-*.html   # listing pages
│   ├── cold-room.html  customized-packaging.html  contact.html  returns-policy.html
│   ├── sw.js                     # staff Web Push service worker
│   ├── assets/css/               # COMMITTED compiled Tailwind (storefront + admin), RTL/dark overrides
│   ├── assets/js/                # i18n.js + i18n-data.js, checkout.js, account.js, cart-nav.js,
│   │                             #   search.js, filters, nav, locations data, vendor/libphonenumber
│   ├── assets/fonts/             # self-hosted fonts
│   ├── uploads/                  # admin-uploaded product images (~52 files, ~35 MB) — live data
│   ├── topcar/ advertisment/ categories/   # site imagery
```

```
│   └── (~15 superseded variants) cart2-7.html, checkout2-4.html, home.html, houseware2.html…
├── whatsapp-service/            # STANDALONE Baileys service (separate npm project)
│   ├── src/                     # config, server, whatsapp (socket lifecycle), sender, auth, test-ui
│   ├── dist/                    # compiled JS (npm run build = tsc)
│   ├── auth/                    # ⚠ LIVE WHATSAPP SESSION — never commit/share/copy
│   ├── .env                     # PORT, HOST, API_KEY, AUTH_DIR, timeouts (git-ignored)
│   └── package.json             # baileys 7.0.0-rc14, express, pino, qrcode; node >= 20
├── scripts/                     # operational/maintenance scripts (Node .mjs/.cjs)
│   ├── wa-service-manager.mjs   # starts/stops the WhatsApp service for `next dev`
│   ├── create-store-orders-staff.mjs, seed-owner-test-data.mjs, cleanup-owner-test-data.mjs
│   └── (catalog/warehouse migration + audit helpers)
├── docs/                        # historical planning/audit documents (may be outdated)
├── middleware.ts                # edge: page gates, rate limits, Origin check, headers
├── instrumentation.js           # auto-starts the WhatsApp service with `next dev`
├── next.config.mjs              # rewrites, redirects, cache headers, CSP
├── neon.ts                      # Neon branch policy
├── tailwind.config.js / tailwind.admin.config.js / *.src.css   # Tailwind build inputs
├── DATABASE_SETUP.md            # Neon configuration record (accurate)
├── AGENTS.md, CHANGELOG.md, docs/*                            # historical docs
└── _backup-pre-*/  _nav-backup/  code backup/  db-backups/  backups/
    gui-test-screenshots/  certs/  .playwright-mcp/  rackingpage/   # NOT part of the app
```

Notes:

- `rackingpage/` is an isolated React experiment (warehouse racking page) not wired into routing; `certs/` holds LAN dev HTTPS keys (git-ignored); `db-backups/` holds Postgres dumps and screenshots (git-ignored).
- Loose root files (`*.cjs` helpers, `order-backup-*.json`, diagnostic PNGs, `nassim_backup.dump`, `dev-server.log`) are working artifacts, not application code.

---

4. Database Documentation

PostgreSQL 18 on Neon. Prisma schema: `prisma/schema.prisma` (637 lines, **38 models, 8 enums**). All integer PKs are `autoincrement` unless noted.

4.1 Enums

Enum	Values
`PurchaseMode`	ONLINE, INQUIRY (per Department; **no storefront behaviour consumes INQUIRY yet**)
`UserRole`	CUSTOMER, STAFF, ADMIN, SUPERADMIN
`OrderStatus`	PENDING, SEEN, UNDER_REVIEW, READY_FOR_CONFIRMATION, CONFIRMED, PACKING, READY_FOR_DELIVERY, OUT_FOR_DELIVERY, DELIVERED, COMPLETED, CANCELLED_BY_CUSTOMER, CANCELLED_BY_STAFF
`OrderItemStatus`	PENDING, AVAILABLE, UNAVAILABLE, REMOVED_AFTER_CONFIRMATION, DELIVERED
`PosStatus`	NOT_CREATED, INVOICED, HANDED_TO_DELIVERY
`StockStatus`	IN_STOCK, LOW_STOCK, OUT_OF_STOCK, PREORDER, DISCONTINUED
`AttributeSection`	HEADER, DESCRIPTION, SPECIFICATIONS
`AttributeType`	TEXT, LONG_TEXT, NUMBER, SELECT

4.2 Catalogue / PIM models

Model	PK	Purpose / key relations
`Department`	id	Top-level department; `slug` unique; `purchaseMode` (ONLINE/INQUIRY). Has Sections and Categories.
`Section`	id	Optional grouping inside a Department (FK `departmentId`).
`Category`	id	FK `departmentId`, optional FK `sectionId`; `slug` unique; holds Subcategories and Products.
`Subcategory`	id	FK `categoryId`; holds Products and `CustomAttribute` definitions.
`Product`	id	The central entity. FKs: `categoryId` (required), `subcategoryId?`, `brandId?`, `materialId?`, `supplierId?`, `unitId?`, `countryId?`, `taxId?`. Money: `price`, `discount`, `costPrice` Decimal(10,3), `currency` default "KD". Bilingual: `name` + `nameAr` (empty ⇒ fallback to English). Identifiers: `slug` (unique), `sku`, `barcode`, `ndNumber`, `internalCode`, `classCode` (internal variant-group code, **never exposed publicly**). Flags: `isActive` (publish), `isFeatured`, `isNewArrival`, `isBestSeller`. Stock: `stock`, `minStock`, derived `stockStatus`. Media: `image` (string URL) + `images` (string[]). Specs: `specs Json`. Dimensions: `weight/length/width/height/depth` Decimal(10,3) + `dimensionUnit` (default "cm"). SEO: `seoTitle`, `seoDescription`, `tags[]`. Timestamps.
`Brand` `Material` (+ `MaterialI18n`) `Color` (+ `ColorI18n`) `Size` `Supplier` `Unit` `Country` `Tax`	id	Master-data tables (all with unique slugs). Color has `hex`; Tax has `rate` Decimal(6,3). Colors/Sizes link to products via join tables.
`ProductColor`, `ProductSize`	composite (`productId`, `colorId` / `sizeId`)	M:N join tables, cascade delete.
`ProductImage`	id	Ordered media rows (`url`, `sortOrder`, `isPrimary`) FK `productId` cascade. Written only by restore tooling; the live admin uses `Product.image/images`.
`CustomAttribute`	id	Dynamic field **definition** per Subcategory (FK) or Category fallback (`categoryId`, natural id, no FK). Fields: `name`, `section`, `fieldType`, `options[]`, `displayOrder`, `isDeleted` (soft delete).
`AttributeI18n`	attributeId (PK, no FK)	Arabic name + `multiSelect` for SELECT attributes.
`AttributeOptionI18n`	id	Arabic label per option (unique `[attributeId, optionEn]`).
`ProductCustomValue`	id	Per-product value, unique `[productId, attributeId]`, FK cascade both.
`ProductCustomValueI18n`	composite PK (no FKs)	Arabic value for a product's custom field.
`RelatedProduct`	id	Admin-curated related products: `productId` → `relatedId` (both FK cascade), unique pair, `displayOrder`. Storefront falls back to same-section products when absent.
`StockMovement`	id	Stock ledger rows (`delta`, `reason`, `note`, `actorId?`) FK `productId` cascade. Written by order flows; **no admin endpoint manages it**.

4.3 Users & sessions

Model	PK	Purpose / key fields
`User`	id	`email` unique, `name`, `phone`, `passwordHash` (bcrypt cost 12), `role` (enum), `designation` (free-text label for STAFF — grants no permissions), `isActive`. Relations: sessions, orders, addresses, wishlist, cartItems, passwordTokens.
`Session`	id (uuid string)	FK `userId` cascade; `expiresAt`. DB-backed sessions — source of truth for validity.
`Address`	id	FK `userId` cascade; label, fullName, phone, address, city, area, `isDefault`.
`CartItem`	id	Unique `[userId, productId]`; `quantity`. **POST semantics: sets (not increments) quantity.**

Model	PK	Purpose / key fields
`WishlistItem`	id	Unique `[userId, productId]`.
`PasswordResetToken`	id	FK `userId` cascade; `tokenHash` unique (SHA-256 of a 32-byte random token), `expiresAt` (30 min), `usedAt`.

4.4 Orders

Model	PK	Purpose / key fields
`Order`	id	`orderNumber` unique (`ORD-<n>` from Postgres sequence `order_number_seq`). Customer snapshot: `customerName/Email/Phone`, `address`, `city`, `notes`. `status` (enum), `staffNotes`, `posStatus`. Money: `subtotal`, `shipping` (default 2.5), `total`, `currency` "KD". **Permanent ownership snapshot** `originalUserId?` / `originalUserEmail` (survive account deletion; live `userId?` / `customerEmail` may be tombstoned). **WhatsApp tracking**: `whatsappSent` bool, `whatsappStatus` text default "PENDING", `whatsappAuto` bool, `whatsappSentAt?`, `whatsappError?`. Relations: `items`, `statusHistory`, `notifications`. Many indexes incl. `originalUserEmail`, `createdAt`.
`OrderItem`	id	FK `orderId` cascade; `productId?` (nullable — survives product changes), plus snapshot fields `slug`, `name`, `image`, `price` Decimal(10,3), `quantity`, `itemStatus` (enum).
`OrderSecurity`	`orderNumber` (string PK, **no FK** by design — legacy Order table owned by migration role)	Guest-access security: `tokenHash` unique (SHA-256 of the one-time tracking token), `idempotencyKey?` unique (checkout replay guard).
`OrderStatusEvent`	id	FK `orderId` cascade; append-only status history (`status`, `note`, `createdAt`).
`Notification`	id	In-app notifications; optional FKs `userId?`, `orderId?` cascade; `type`, `message`, `readAt`.

4.5 Settings & audit

Model	PK	Purpose
`Setting`	`key` (string)	Generic key/value store. Known keys: `shipping_threshold`, `shipping_fee_below_threshold`, `shipping_fee_at_or_above`, `whatsapp_automation_enabled` (default off), `whatsapp_test_mode` (**on** unless value is exactly `"false"`), plus JSON rows holding Web-Push subscriptions (`lib/web-push.ts`).
`AuditLog`	id	`actorId?`, `action` (e.g. PRODUCT_CREATE, ORDER_STATUS_CHANGE, WHATSAPP_AUTO_SENT, STAFF_PASSWORD_RESET…), `entity`, `entityId`, `detail`, `createdAt`. The de-facto product change history.

4.6 Entity relationship summary

```
Department 1—* Section (optional)
Department 1—* Category *—1 Department, *—1 Section(optional)
Category   1—* Subcategory
Product    *—1 Category (required), *—1 Subcategory(optional), *—1 Brand/Material/
           Supplier/Unit/Country/Tax(optional)
Product 1—* ProductImage / StockMovement / CartItem / WishlistItem / OrderItem(optional FK)
Product *—* Color via ProductColor ; *—* Size via ProductSize
Subcategory|Category 1—* CustomAttribute 1—* ProductCustomValue *—1 Product
Product 1—* RelatedProduct(from) ; Product 1—* RelatedProduct(to)
User 1—* Session / Address / CartItem / WishlistItem / PasswordResetToken / Order
Order 1—* OrderItem (?—1 Product) ; 1—* OrderStatusEvent ; 1—* Notification ; 1—1 OrderSecurity (no FK)
```

Deliberate FK-less natural-id tables (documented in the schema): `OrderSecurity`, `AttributeI18n`, `AttributeOptionI18n`, `ProductCustomValueI18n`, `ColorI18n`, `MaterialI18n`, and `CustomAttribute.categoryId`.

4.7 Migrations (18)

`20260720164813_init` → `phase2_erp_master_data` → `sprint2_order_verification_workflow` → `password_reset_tokens` → `order_ownership_snapshot` → `dynamic_attributes_related_products` → `add_superadmin_role_owner_dashboard` → `security_order_access_and_idempotency` → `dynamic_spec_i18n_tables` → `add_product_class_code` → `custom_attribute_category_fallback` → `add_depth_and_dimension_unit_to_product` → `add_order_status_seen` → `add_order_whatsapp_sent` → `add_product_name_ar` → `order_number_sequence` (creates `order_number_seq`) → `add_whatsapp_notification_status` (2026-11-09, latest).

Operational rule (from `DATABASE_SETUP.md`): use `prisma migrate deploy` against `DIRECT_URL`; never `migrate reset` / `db push` on production.

---

5. Main Features and Functional Modules

Status key: **IMPLEMENTED** = verified working code end-to-end; **PARTIAL** = present but incomplete; **NOT FOUND** = absent from the code.

5.1 Customer storefront

Feature	Status	Notes (verified)
Homepage & departments	IMPLEMENTED	`home5.html` (rewritten from `/`); departments: Houseware (Kitchenware / Cleaning Tools / Home & Outdoor), Supermarket (5 subcategory pages), Warehouse (3 pages), plus Cold Room and Customized Packaging landing pages. Data via `GET /api/catalog`.
Product listings, filtering, sorting, pagination	IMPLEMENTED (client-side)	All listing pages render from the catalog payload; `subcategory-filter.js`, `color-filter.js`, `price-range-slider.js`, `listing-search-filter.js`, `sort-dropdown.js`, `listing-pagination.js`. No server-side pagination for customers.
Product detail	IMPLEMENTED	`product-view.html?id=<slug>`; gallery, variants via `classCode` siblings, custom attributes (bilingual), related products, add-to-cart/wishlist.
Search	IMPLEMENTED	`search.js` overlay; product **name** (EN/AR) matching only; all-terms then first-term fallback.
Cart	IMPLEMENTED	Guest: localStorage `nassim_cart`. Logged-in: server `/api/cart` (sets quantity, not increments). Merge at login (`account.js`).
Wishlist	IMPLEMENTED	Same pattern via `/api/wishlist`.
Checkout	IMPLEMENTED	`checkout5.html` + `checkout.js`: name, mobile (country selector, default Kuwait +965, vendored libphonenumber validation), optional email, Country/Governorate/Area (6 governorates / 134 areas of Kuwait), block/street, notes; saved addresses autofill; shipping fee display from `/api/shipping`; draft autosave.
Order submission	IMPLEMENTED	`POST /api/orders` with idempotency key; server-side prices; guest tracking token; `ORD-<n>` numbering.
Order-sent sheet	IMPLEMENTED	`order-send-modified4.html` (served `no-store`; one-shot view).
Order tracking	IMPLEMENTED	`order-detail.html?number=…` (+ token) and the signed `/o/{id}-{hmac16}` short link served by `app/o/[slug]/route.ts` into `customer-order.html`.
Customer accounts	IMPLEMENTED	Register / login / logout / profile update / password reset (Resend email) / delete account (orders preserved via ownership snapshot + tombstoning) / addresses.
Arabic & English	IMPLEMENTED	`NassimLang` toggle; preference `localStorage.nassim-lang`; translations in `i18n-data.js` (generated from an xlsx pipeline); text-level RTL; DB-level bilingual product data (`nameAr`, i18n side tables) with English fallback.
INQUIRY purchase mode	PARTIAL	Enum + admin editing exist; **no storefront behaviour consumes it**.
PWA / offline	PARTIAL	`sw.js` is staff push only; no manifest, no offline caching, no robots.txt/sitemap.

5.2 Admin & staff

Feature	Status	Notes
Staff authentication	IMPLEMENTED	Shared session system; edge-gated pages; STAFF vs ADMIN server-enforced; login rate-limited.

Feature	Status	Notes
Order management	IMPLEMENTED	List with filters (status, text, date range) + pagination; status state machine with row locking; per-item verify; pack chain (pack/ready/deliver/complete) + POS status; timeline; signed order link; staff Web Push + sound alerts.
Order status updates	IMPLEMENTED	Full transition table (§2.7); illegal/raced transitions → 409; cancellations restore reserved stock.
Product management	IMPLEMENTED	Full CRUD (soft delete only), images, custom fields, related products, flags, stock with audit; catalog cache invalidation.
Category & hierarchy management	IMPLEMENTED	`hierarchy/[entity]` (departments incl. purchaseMode, sections, categories, subcategories) + `master/[entity]` (brands, materials, colors, sizes, suppliers, units, countries); hard delete with in-use guard. Taxes CRUD deliberately removed.
Customer management	IMPLEMENTED	Search/list with counts (STAFF); detail with orders (STAFF); activate/deactivate (ADMIN) with last-admin guard; no delete.
Backup & restore	IMPLEMENTED	ZIP downloads (orders JSON; products JSON + entire `public/uploads` + manifest with missing-image report) and non-destructive merge restore with `?dryRun=1`, chunked upload, images never overwritten. Restore is **not transactional** and writes no audit rows.
Staff management	IMPLEMENTED	Create/edit/deactivate/password-reset STAFF accounts; designation is a label only; no delete.
Admin tools	IMPLEMENTED	Dashboard stats, low-stock view, audit log browser, printable product report (client-side print → PDF), WhatsApp automation settings.
Charts on dashboard	NOT FOUND	`recharts` is an unused dependency; dashboard is cards/lists.
Bulk edit / Excel import-export	NOT FOUND	`exceljs` unused; report is JSON + client rendering.

5.3 PIM capability matrix

Capability	Status
Product create/edit, bilingual fields, dynamic custom attributes, related products, variant grouping (`classCode`), publish/unpublish (`isActive`), per-product min-stock, image upload/optimisation	IMPLEMENTED
Duplicate detection	PARTIAL — slug uniqueness + form pre-checks only; no SKU/barcode duplicate report
Versioning / change history	PARTIAL — `AuditLog` (incl. `PRODUCT_STOCK_CHANGE` old→new); no version table/rollback
Data-quality scoring	NOT FOUND (only stock indicators)
Import (Excel/CSV/feed)	NOT FOUND as a feature (Google Shopping feed images exist historically; import was scripted, `scripts/` contains one-off migration helpers)
Bulk operations	NOT FOUND beyond a bulk-activate action in `ProductsTab.tsx`
Stock movements ledger	PARTIAL — maintained by order flows; no admin UI/endpoint to manage it

5.4 WhatsApp

See also §2.9, §6.9 and §7. Each item below was verified in `whatsapp-service/src/` , `lib/whatsapp-automation.ts` , `instrumentation.js` and `scripts/wa-service-manager.mjs` .

Baileys service HTTP API (listens on `127.0.0.1:4000` in the integrated setup; not reachable from browsers — only the Next.js server calls it):

Method & path	Auth	Purpose
GET `/health`	none	Liveness: `{ok, uptime}` . Polled by the service manager at startup.
GET `/status`	none	Connection state (e.g. `awaiting_qr` , `open`) and the logged-in account id.

Method & path	Auth	Purpose
GET `/qr`	API key	Current pairing QR as a PNG data URL; 404 when already connected or no QR yet. QR rotates roughly every 20 s.
POST `/logout`	API key	Unlinks the device, wipes the `auth/` session, restarts the socket so a fresh QR is issued.
POST `/send-message`	API key	Body `{phone, message, idempotencyKey?}` → `{ok, delivered, messageId, maskedPhone, idempotentReplay}`. Errors: 400 `INVALID_PHONE` / `INVALID_MESSAGE`, 503 `NOT_CONNECTED`. JSON body limit 16 kb.
`/test`, `/test/*`	none	Development-only test UI (30-second scheduled test send). Localhost-bound; do not expose.

Phone numbers are normalised (digits only, 8–15 digits) and addressed as `<number>@s.whatsapp.net`; there is **no "is this number on WhatsApp" check** before sending, and "delivered" means WhatsApp accepted the message, not that it was read. There is no queue or rate limiter: one message per HTTP call. Logs mask phone numbers and do not include message bodies.

Capabilities:

- **Baileys service architecture** — IMPLEMENTED: standalone Express/TypeScript process, one WhatsApp session, API-key protected, session persisted in `whatsapp-service/auth/` (multi-file auth state).
- **Automatic customer notification after checkout** — IMPLEMENTED (fire-and-forget; gated by `whatsapp_automation_enabled`, default **off**).
- **Manual staff notification** — IMPLEMENTED (`POST /api/whatsapp/send`, any staff).
- **TEST MODE** — IMPLEMENTED: `whatsapp_test_mode` Setting is **on unless explicitly** `"false"`; while on, every recipient not on the `WHATSAPP_TEST_RECIPIENT` allow-list is redirected to the first allow-listed number. Toggled by ADMIN in Settings; going live needs no code change.
- **API key authentication** — IMPLEMENTED (`Authorization: Bearer` / `X-API-Key`; plain string comparison).
- **Connection & QR management** — IMPLEMENTED: the "LINK WHATSAPP" panel on `internal-orders2.html` polls `/api/whatsapp/status` (every 3 s) and refreshes `/api/whatsapp/qr` (~20 s); disconnect via `/api/whatsapp/disconnect` after a confirmation dialog (wipes session, new QR).
- **Session persistence** — IMPLEMENTED: multi-file auth state in `whatsapp-service/auth/`; survives restarts; exponential reconnect backoff capped at `MAX_RECONNECT_ATTEMPTS` (default 10).
- **Service startup/shutdown** — IMPLEMENTED: `npx next dev` auto-starts it via `instrumentation.js` → `scripts/wa-service-manager.mjs` (port-4000 busy check, health poll, tree-kill on exit). Production `next start` does **not** auto-start it unless `WA_SERVICE_AUTO_START=1`.
- **Notification status & duplicate protection** — IMPLEMENTED: `PENDING→SENDING→SENT/FAILED` state machine with atomic claim + 2-min stale expiry, plus a 24-h service-side idempotency key; FAILED orders can be retried manually.

5.5 Feature completeness snapshot

Fully implemented: storefront browsing/search/cart/checkout/guest+member ordering, order tracking (token + signed link), accounts incl. self-service deletion, order pipeline with verification workflow, catalogue & master-data admin, uploads, backups/restores, staff push, password reset email, WhatsApp notifications with TEST MODE.

Partial: INQUIRY mode (server only), duplicate detection, change history (audit log only), stock-movement management, PWA/offline, super-admin concept (enum exists, no UI/access).

Not implemented: bulk edit, Excel import/export, product version table, data-quality tooling, customer-facing offline/PWA, charts, robots/sitemap, CI/tests.

---

6. API Documentation

All endpoints are same-origin (`/api/...`). Envelope: `{ success, data, error }`. Auth legend: 🌐 public · 👤 customer session · 👥 STAFF or ADMIN (`requireStaff`) · 🔒 ADMIN only (`requireAdmin`). CSRF protection for all mutating calls = `SameSite=Strict` cookie + middleware Origin/Host check (403 on cross-origin). `JWT_SECRET`, keys, tokens: never exposed here.

6.1 Authentication

Method & path	Auth	Purpose / request / response
POST `/api/auth/login` 🌐	none (rate-limited 10/min/IP)	Body `{email, password}` (zod). 200 `{data:{user:{id,name,email,phone,role}}}` ; 401 generic invalid/inactive; sets session cookie; creates `Session` row.
POST `/api/auth/logout` 🌐	none	Destroys session row + cookie. Always 200.
GET `/api/auth/me` 🌐	optional	200 `{data:{user, guest}}` — `user:null, guest:true` when anonymous.
PATCH `/api/auth/me` 👤	session	Body `{name?, email?, phone?, password?, currentPassword?}` ; email/password changes require `currentPassword` ; email collision 409; password change rotates the current session only.
POST `/api/auth/register` 🌐	rate-limited	Body `{name, email, password, phone?}` ; creates CUSTOMER + auto-login; 409 duplicate email.
DELETE `/api/auth/delete-account` 👤	session + password re-entry (rate limit 5/10 min)	Tombstones orders (`userId:null` , email → `deleted+<id>+<email>`), deletes user + cascade, destroys session. Permanent ownership snapshot (`originalUserId/originalUserEmail`) keeps orders identifiable.
POST `/api/auth/password-reset/request` 🌐	rate limit 3/10 min/IP	Body `{email}` ; always 200 (anti-enumeration); creates hashed single-use token (30 min) and emails link via Resend.
POST `/api/auth/password-reset/reset` 🌐	token capability	Body `{token, password, confirm}` ; marks token used; **deletes all sessions** (forced re-login).

6.2 Catalogue (public reads)

Method & path	Purpose / notes
GET `/api/products` 🌐	Paginated active-product search: `q` (name/description/sku/line), `category` slug, `page` , `limit` (1–100, default 24). Returns product cards incl. `purchaseMode` , `customFields` .
GET `/api/products/[slug]` 🌐	Single active product; 404 if inactive/missing; adds category/department fields, customer-safe `variantOptions` (from `classCode` siblings — `classCode` itself never exposed), custom fields.
GET `/api/catalog` 🌐	**JavaScript payload** defining `window.NASSIM_PRODUCTS` + `NassimGetProduct()` for all active products (colors, variants, custom fields). Cached in-process + 10 s client max-age. The storefront's primary data source.
GET `/api/categories` 🌐	Departments → categories with active product counts.
GET `/api/shipping` 🌐	Display-only shipping rule from `Setting` keys (defaults: threshold 20 KD, 1 KD below, free at/above). Server recomputes authoritatively at order creation.

6.3 Customer data (session required)

Method & path	Purpose / notes
GET/POST/DELETE `/api/cart` 👤	List; add/set `{slug, quantity 1–999}` (**sets** quantity); delete one (`?slug=`) or all.
GET/POST/DELETE `/api/wishlist` 👤	Same pattern; POST idempotent `{slug}` .
GET/POST `/api/addresses` 👤	List (default-first); create `{label?, fullName?, phone?, address, city, area?, isDefault?}` .
GET/PATCH `/api/notifications` 👤	Last 50 notifications; PATCH marks one (`{id}`) or all (`{all:true}`) read. Untyped body — silently no-ops if neither field present.

6.4 Orders

Method & path	Auth	Purpose / notes
POST `/api/orders` 🌐	none (rate limit 8/10 min/IP; Origin check)	Place order. Body: customerName, customerEmail (optional), customerPhone, address, city, notes?, items[1–100] {slug,name,image?,price,qty}, currency?. **Prices recomputed from DB; shipping computed server-side.** Optional `x-idempotency-key` header (8–80 chars). Creates order (`ORD-<n>`), items, first history event, `OrderSecurity` (tokenHash + idempotencyKey), cleans server cart for logged-in users. Response includes the **one-time** `accessToken` and `shortPath` (`/o/<id>-<sig>`). Replay returns the original order with `accessToken:null` . Triggers staff Web Push + automatic WhatsApp (fire-and-forget).
GET `/api/orders` 👤	session	The user's own orders (by `userId` only) newest-first with billable-total recomputation (non-billable item statuses excluded). Payload under both `orders` and `data` (legacy).
GET `/api/orders/[id]`	owner 👤 / staff 👥 / guest 🌐 + `?token=`	Order detail incl. items, item statuses, timeline, staff notes. `?by=number` supported. Guests must present the raw capability token.
POST `/api/orders/[id]/confirm`	owner 👤 / guest 🌐 + `?token=`	Body `{action:"confirm"|"cancel"}` → CONFIRMED / CANCELLED_BY_CUSTOMER; row-locked transition (409 on illegal/raced); cancel restores reserved stock; confirm removes unavailable items and restores their stock; creates in-app notification.

6.5 Admin — orders

Method & path	Auth	Purpose / notes
GET `/api/admin/orders` 👥	staff	Filters: `status` , `q` (number/name), `page` , `limit` (≤100), `from` / `to` dates. Returns recomputed billable totals, `code` (product SKU), signed `shortPath` , `orderBaseUrl` .
PATCH `/api/admin/orders` 👥	staff	Body `{id, status?, note?, whatsappSent?}` . Status moves validated by the locked state machine (incl. `SEEN`); staff cancellation restores stock; writes `OrderStatusEvent` , audit `ORDER_STATUS_CHANGE` , customer notification. A `whatsappSent` -only PATCH is a pure flag update.
PATCH `/api/admin/orders/[id]/verify` 👥	staff	Body `{items:[{itemId, itemStatus: AVAILABLE|UNAVAILABLE}], staffNotes?}` ; order must be PENDING/UNDER_REVIEW; auto-advances PENDING→UNDER_REVIEW.
PATCH `/api/admin/orders/[id]/pack` 👥	staff	Body `{action: pack|ready|deliver|complete, posStatus?}` → PACKING / READY_FOR_DELIVERY / OUT_FOR_DELIVERY / DELIVERED (+ item → DELIVERED); optional POS status.
GET `/api/admin/orders/[id]/timeline` 👥	staff	Full status history.
POST `/api/admin/orders/[id]/order-link` 👥	staff	Returns the signed `/o/{id}-{hmac16}` link (HMAC-SHA256 of id with `JWT_SECRET`). No DB write.

6.6 Admin — catalogue, master data, PIM

Method & path	Auth	Purpose / notes
GET/POST/PATCH/DELETE `/api/admin/products` 🔒	admin	GET: full list (no pagination; `?id=` for the edit form; `?full=1` legacy serializer) + categories. POST/PATCH: complete zod product schema (names EN/AR, price, stock/minStock, identifiers, `classCode` , specs JSON, dims, flags, relations, colors/sizes, `customValues` ≤200 bilingual, `relatedProductIds` ≤50, image/images); duplicate slug 409; stock changes audited (`PRODUCT_STOCK_CHANGE`); invalidates catalog cache. DELETE `?id=` : **soft delete** (`isActive:false` , audit `PRODUCT_DEACTIVATE`).
GET `/api/admin/products/report` 🔒	admin	JSON dataset for the printable report: `visibility` , `q` , `categoryId` , `idFrom/idTo` ; includes per-product changed-field list derived from audit rows. No file is generated server-side.
GET/POST/PATCH/DELETE `/api/admin/hierarchy/[entity]` 🔒	admin	entity ∈ `departments` (incl. `purchaseMode`), `sections` , `categories` , `subcategories` . Slug-validated CRUD; hard delete blocked when in use (409).
GET/POST/PATCH/DELETE `/api/admin/master/[entity]` 🔒	admin	entity ∈ `brands, materials, colors, sizes, suppliers, units, countries` (+ AR names for colors/materials). Taxes deliberately removed.

Method & path	Auth	Purpose / notes
GET/POST/PATCH/DELETE `/api/admin/attributes` 🔒	admin	Dynamic custom-attribute definitions per `subcategoryId` XOR `categoryId`; SELECT needs ≥1 option; duplicate name in scope 409; DELETE = soft delete.
POST `/api/admin/attributes/reorder` 🔒	admin	`{scope, section, orderedIds}` → `displayOrder` in one transaction.
POST `/api/admin/upload` 🔒	admin	Multipart images ≤8 MB each, magic-byte verified; sharp → resize ≤1600px + WebP q82; saved to `public/uploads/`; returns `{urls:["/uploads/…"]}`. No DB write.
GET `/api/admin/low-stock` 👥	staff	Active products at/below their `minStock`, worst first, with counts.
GET `/api/admin/stats` 👥	staff	Dashboard aggregates: totals, per-status counts, today's pipeline, 8 recent orders, top-10 products, unread notifications for the caller.

6.7 Admin — customers, staff, audit

Method & path	Auth	Purpose / notes
GET `/api/admin/customers` 👥	staff	`q`, `role` filters; counts of orders/addresses/cart/wishlist. No pagination.
GET `/api/admin/customers/[id]` 👥	staff	User + addresses + orders (matched by id **or** email) with timelines.
PATCH `/api/admin/customers/[id]` 🔒	admin	`{isActive}` only. Guards: SUPERADMIN targets 403, self-toggle 409, last-active-ADMIN protected (row-locked); deactivation deletes sessions.
GET/POST `/api/admin/staff` 🔒	admin	List/create STAFF accounts (role hardcoded STAFF; password 8–128 bcrypt-12; audit `STAFF_CREATE`).
GET/PATCH `/api/admin/staff/[id]` 🔒	admin	Edit/deactivate/password-reset; resets and deactivation revoke the target's sessions; granular audits. **No delete.**
GET `/api/admin/audit-log` 🔒	admin	Search/filter/paginate the audit trail with actor names.

6.8 Backup & restore

Method & path	Auth	Purpose / notes
GET `/api/admin/backup/orders` 🔒	admin	ZIP: all orders + items + history + notifications + `OrderSecurity` (hashes only). Filename `nassim-full-orders-backup-YYYY-MM-DD.zip`.
GET `/api/admin/backup/products` 🔒	admin	ZIP: all products with full relation trees + the **entire** `public/uploads/` **tree** + manifest incl. missing-image report.
POST `/api/admin/restore/orders` 🔒	admin	Multipart / raw / **chunked** (`?uploadId&chunk&last=1`) ZIP upload; `?dryRun=1` validates without writing. **Non-destructive merge**: conflicts skipped & reported; never updates/deletes existing rows. Not transactional; no audit rows.
POST `/api/admin/restore/products` 🔒	admin	Same input pattern. Skips existing id/slug; requires `categoryId` to exist; images written to `public/uploads` never overwriting (`wx` flag).

6.9 WhatsApp (Next.js side — proxies to the Baileys service)

Method & path	Auth	Purpose / notes
GET `/api/whatsapp/config` 👥	staff	Automation enabled?, TEST MODE on?, service connected? — drives the staff UI.
GET `/api/whatsapp/status` 👥	staff	Connection state from the service.
GET `/api/whatsapp/qr` 👥	staff	Current QR as data-URL (proxied).
POST `/api/whatsapp/disconnect` 👥	staff	Logs out, wipes the service session, new QR.

Method & path	Auth	Purpose / notes
POST `/api/whatsapp/send` 👥	staff	Body `{orderId}` → manual notification via the same pipeline; 409 ALREADY_SENT / IN_PROGRESS; 403 skipped; 502 failed.
GET `/api/admin/whatsapp-automation` 🔒	admin	Current automation and TEST MODE settings for the admin Settings tab.
POST `/api/admin/whatsapp-automation` 🔒	admin	Body `{enabled?}` and/or `{testMode?}`; persists `Setting` rows; audited (`WHATSAPP_TEST_MODE_TOGGLE` etc.).

(The Baileys service's own endpoints — `/health`, `/status`, `/qr`, `/logout`, `/send-message` — are documented in §5.4; they listen on 127.0.0.1:4000 and are not reachable from browsers.)

6.10 Staff Web Push

Method & path	Auth	Purpose / notes
GET `/api/staff/push` 👥	staff	`{publicKey}` (VAPID public); 503 if unconfigured.
POST `/api/staff/push` 👥	staff	Register/refresh device subscription `{endpoint, keys{p256dh, auth}}`.
PATCH `/api/staff/push` 👥	staff	Pause/resume own device `{endpoint, paused}`.
DELETE `/api/staff/push` 👥	staff	Remove own subscription `{endpoint}`.

---

7. Environment Configuration

Two env files exist: root `.env` (Next.js app; git-ignored) and `whatsapp-service/.env` (service; git-ignored). `.env.example` at the root documents most variables. Values below are placeholders only.

7.1 Root `.env`

Variable	Purpose	Required	Example format
`DATABASE_URL`	Runtime Postgres connection (Neon **pooled** string, `sslmode=require`). Validated at startup by `lib/env.ts` (must start `postgresql://`).	Yes	`postgresql://<user>:<password>@<host-pooler>/<db>?sslmode=require`
`DIRECT_URL`	Prisma migrations connection (Neon **direct/unpooled** string). Referenced as `directUrl` in `schema.prisma`.	Yes (for migrations)	same format, non-pooler host
`DATABASE_URL_UNPOOLED`	Unpooled string kept for reference/tooling (present in current `.env`; not read by app code).	No	same format
`NEON_BRANCH`	Neon CLI branch context (present in current `.env`).	No	branch name
`JWT_SECRET`	HS256 signing key for session JWTs, CSRF HMAC, and `/o/` link signatures. Min 32 chars; validated at startup.	Yes	64+ hex chars
`SESSION_COOKIE_NAME`	Session cookie name. Validated at startup.	Yes	`nassim_sid`
`ADMIN_SEED_EMAIL` / `ADMIN_SEED_PASSWORD`	Seeded ADMIN account (validated; password never re-hashed if user exists).	Email yes; password for seeding	emails / strong passwords
`SUPERADMIN_SEED_EMAIL` / `SUPERADMIN_SEED_PASSWORD`	Seeded owner account (note: SUPERADMIN role exists but is admitted nowhere at runtime).	No (optional)	—
`STAFF_SEED_EMAIL` / `STAFF_SEED_PASSWORD`	Seeded STAFF account.	No (optional)	—
`STORE_ORDERS_STAFF_EMAIL` / `STORE_ORDERS_STAFF_PASSWORD`	Dedicated store-orders staff account, created by `scripts/create-store-orders-staff.mjs`.	No (optional)	—

Variable	Purpose	Required	Example format
`RESEND_API_KEY`	Resend API key for password-reset emails.	Yes (for resets)	`re_...`
`EMAIL_FROM`	From address for reset emails.	Yes (for resets)	`AL-NASSIM <no-reply@domain>`
`APP_URL`	Origin used e.g. for order links/admin links (present in `.env.example`).	Recommended	`http://localhost:3000`
`APP_BASE_URL`	**Public** origin embedded in WhatsApp order links (must be the reachable production origin).	Recommended	`https://<domain>`
`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` / `VAPID_SUBJECT`	Web Push (staff new-order notifications). Server-side only.	For push	Base64URL keys; `mailto:` subject
`WHATSAPP_SERVICE_URL`	Base URL of the Baileys service. Default coded: `http://127.0.0.1:4000`.	Yes (for WhatsApp)	`http://127.0.0.1:4000`
`WHATSAPP_API_KEY`	Shared secret sent to the service as `X-API-Key` /Bearer. Must equal the service's `API_KEY`.	Yes (for WhatsApp)	random string
`WHATSAPP_TEST_RECIPIENT`	Comma-separated allow-list for TEST MODE; any other recipient is redirected to the first entry. Hard-coded fallback exists in `lib/whatsapp-automation.ts`.	Recommended	`<dialling-code-full-number>,<number>`

Optional, coded defaults (not required): `IMAGE_MAX_WIDTH` (upload resize, default 1600), `IMAGE_WEBP_QUALITY` (default 82), `NEXT_DIST_DIR` (build dir override), `WA_SERVICE_AUTO_START` (=1 makes `next start` / `next build` also auto-start the WhatsApp service), `NASSIM_WA_STARTED` (internal re-entry guard).

7.2 `whatsapp-service/.env` (no `.env.example` ships in that folder — the README references one that is missing)

Variable	Purpose	Required	Default (coded)
`PORT`	HTTP port (manager forces 4000 when started by `next dev`)	No	3000
`HOST`	Bind host (default loopback)	No	`127.0.0.1`
`API_KEY`	Shared secret for `/qr`, `/logout`, `/send-message`	**Yes**	—
`AUTH_DIR`	Baileys session directory (contains live session; git-ignored)	No	`auth`
`SEND_TIMEOUT_MS`	Wait-for-connected timeout per send	No	30000
`IDEMPOTENCY_TTL_MS`	Send-idempotency memory TTL	No	86400000
`MAX_RECONNECT_ATTEMPTS`	Reconnect backoff cap	No	10

---

8. Installation and Local Setup (Windows)

All commands verified against `package.json` scripts and the current machine's working setup.

8.1 Prerequisites

- **Node.js ≥ 20** (Next.js requirement; current dev machine runs **v24.18.1**; the WhatsApp service engines field requires ≥ 20). npm ≥ 10.
- Git.
- A Neon (or any PostgreSQL 14+) database. Neon recommended — `DATABASE_SETUP.md` records the live project.
- No Python, no C compiler toolchain needed beyond what `sharp` ships prebuilt binaries for.

8.2 Install

```
cd "V:\14 SEPT WITH ONLY ORDER\5-8-26\backup 4-8-26\nassim-main"
npm install                # root dependencies (Next.js app)
cd whatsapp-service
npm install                # separate dependency tree for the Baileys service
cd ..
```

8.3 Environment files

1. Copy `.env.example` to `.env` and fill in real values (see §7.1). Minimum for boot: `DATABASE_URL`, `JWT_SECRET` (≥32 chars), `SESSION_COOKIE_NAME`, `ADMIN_SEED_EMAIL`. Startup validation (`lib/env.ts`) exits loudly if these are missing.
2. Create `whatsapp-service/.env` with at least `API_KEY=<random>`; set the same value as `WHATSAPP_API_KEY` in the root `.env`, plus `PORT=4000`, `HOST=127.0.0.1`, `AUTH_DIR=auth`.

8.4 Database setup

```
npx prisma generate          # generate the Prisma client — REQUIRED after every npm install:
                             #   package.json has NO postinstall hook that does this
npx prisma migrate deploy    # apply all 18 migrations (uses DIRECT_URL)
npm run seed                 # node prisma/seed.mjs — creates admin/owner/staff accounts;
                             #   products are NOT seeded (catalog is managed via admin panel)
```

Never run `prisma migrate reset` or `prisma db push` against the production database.

8.5 Development startup (both services)

```
npx next dev
```

This single command starts **both** services (see `instrumentation.js` + `scripts/wa-service-manager.mjs`):

1. WhatsApp Baileys service → `http://127.0.0.1:4000` (skipped automatically if port 4000 is already in use; existing `whatsapp-service/auth/` session is reused, no re-scan needed).
2. Next.js → `http://localhost:3000` (bound `0.0.0.0`).

Service logs are forwarded into the same terminal. Stopping with **Ctrl+C stops both** (the manager tree-kills the service; a safety net also kills it if the Next server crashes).

Equivalent script: `npm run dev` (same behaviour, port 3000 fixed). HTTPS LAN variant: `npm run dev:https` (requires certs from `npm run dev:https:certs`, which uses mkcert and hardcodes LAN IP `192.168.1.172`).

8.6 Running the WhatsApp service manually

```
cd whatsapp-service
npm run build              # tsc -> dist/
npm start                  # node dist/server.js   (reads whatsapp-service/.env)
```

8.7 Verify it is running

- `http://localhost:3000` → homepage (`home5.html`).
- `http://127.0.0.1:4000/health` → `{"ok":true,...}` from the WhatsApp service.
- Admin login at `http://localhost:3000/admin/login` (seeded ADMIN account from `.env`).
- WhatsApp connection: sign in as staff → `http://localhost:3000/internal-orders2.html` → "LINK WHATSAPP" panel → scan the QR once (session then persists in `whatsapp-service/auth/`). The panel only appears when automation is enabled in Admin → Settings.

8.8 Production build & start

```
npm run typecheck           # tsc --noEmit
npm run build               # next build (via scripts/readlink-shim.cjs wrapper)
npm run start               # next start -p 3000
```

`npm run build` is `node --require ./scripts/readlink-shim.cjs ./node_modules/next/dist/bin/next build`. The shim exists because this working copy lives on a volume whose `readlink()` returns `EISDIR` for regular files, which crashes `next build`; it translates that to `EINVAL`. It is harmless on a normal disk, so keep using `npm run build` rather than `npx next build` on this drive.

Note: `next start` does **not** auto-start the WhatsApp service unless `WA_SERVICE_AUTO_START=1`; otherwise start it manually (§8.6) under a process manager (pm2/systemd per the service README). After changing Tailwind classes, rebuild CSS first: `npm run build:css` and/or `npm run build:css:admin` (compiled CSS is committed).

8.9 Rebuilding translations

Storefront strings live in `public/assets/js/i18n-data.js`, generated from an approved xlsx via a `translation-system/apply.cjs` pipeline referenced in `i18n.js`'s header. The xlsx and pipeline location could **not be verified** as present in the repo — treat translations as hand-editable JS if the source file is unavailable.

---

9. Deployment and Infrastructure

Verified facts:

- **Database hosting:** Neon PostgreSQL 18, project "al-nassim website" (aws-us-east-2), branch `production`, db `neondb` (documented in `DATABASE_SETUP.md` and reflected by `DIRECT_URL` / `DATABASE_URL_UNPOOLED` in `.env` and `neon.ts`).
- **File/image storage:** local disk `public/uploads/` only. There is **no cloud object storage**. On any stateless host, uploaded images and the WhatsApp session directory are ephemeral unless a persistent disk or an external store is added.
- **Git remotes** (names only): `origin` → `github.com/iamleo4012/nassim-2.0`, `newweb` → `iamleo4012/new-nweb`, `vercel-deploy` → `leojoseph27/al-nassim` — a Vercel-oriented mirror repository exists.
- `DATABASE_SETUP.md` **contains a "Vercel readiness" checklist** (pooled `DATABASE_URL`, `DIRECT_URL` for migrations, `sslmode=require`, env vars set in the Vercel project, Prisma engines on build). It is explicitly marked **"do not deploy yet"**.

Not verifiable from the source code:

- There is **no `vercel.json`, Dockerfile, PM2 ecosystem file, Procfile, CI workflow (`.github/`), or IaC** anywhere in the project. No deployment pipeline is configured in the repo.
- Whether a live production deployment currently exists (the checklist says production deployment had not happened as of its writing).

Conclusions / constraints a new developer must know:

1. **The Baileys service makes serverless (Vercel functions) impossible as-is**: it needs a long-running Node process, a persistent `auth/` directory, and localhost reachability from the Next server (`WHATSAPP_SERVICE_URL=http://127.0.0.1:4000`). A Vercel deployment of the Next app would lose automatic WhatsApp notifications unless the service is hosted on a companion always-on host (VPS/Fly/Railway-style) and `WHATSAPP_SERVICE_URL` is pointed at it (with the API key protected; the service binds `127.0.0.1` by default and its README suggests a TLS reverse proxy + `HOST=0.0.0.0` on a VPS).
2. `public/uploads` must be persistent; the products backup ZIP exists precisely to move images between hosts (restore never overwrites existing files).
3. In-memory rate limiting and the catalog cache assume a **single Node instance**; multi-instance deployment would need Redis or equivalent (documented in `lib/security.ts`).
4. Services that must run continuously in production: the Next.js server **and** the WhatsApp service (one WhatsApp session; do not run two instances against the same `auth/` directory).
5. Neon compute auto-suspends when idle (first request after suspension is slow — expected).

---

10. Security and Maintenance

10.1 Authentication & access control

- Sessions: HS256 JWT in HttpOnly `SameSite=Strict` cookie + **DB-backed `Session` rows** re-validated on every request; deactivation instantly kills access. 30-day expiry.
- Role gates: `requireStaff()` (ADMIN|STAFF) and `requireAdmin()` on every route; SUPERADMIN not accepted anywhere. Admin/staff *pages* are edge-gated by middleware; APIs always enforce server-side.
- Guards: last-active-ADMIN deactivation protection (row-locked), self-deactivation block, SUPERADMIN-target block, staff-role hardcoding (no escalation path), password re-entry for account deletion.
- Rate limits: login/register 10/min/IP (middleware), password-reset request 3/10 min, order creation 8/10 min/IP, account deletion 5/10 min. In-memory, per-process.

10.2 Password & secret handling

- Passwords: bcrypt cost **12**, centralized in `lib/security.ts`. Generic login errors (no user enumeration); timing-equalised password-reset path.
- Password reset: single-use SHA-256-hashed token, 30-minute expiry, forces re-login everywhere; email via Resend; delivery failure only logged.
- Secrets: all in `.env` files, both git-ignored (verified in `.gitignore`, including `whatsapp-service/.env`, `whatsapp-service/auth/`, `certs/`, `*.dump`, `.claude/`). `lib/env.ts` fails startup on missing required vars. **Historical note from `.gitignore`:** a previous `.claude/settings.local.json` revision leaked database superuser passwords into git history — rotate credentials if cloning from old remotes.

10.3 API security

- CSRF: SameSite=Strict cookie + middleware Origin/Host equality check (403) on all mutating `/api/*` calls with an Origin header. HMAC `x-csrf-token` helpers exist in `lib/security.ts` but are **unused** (defence-in-depth leftovers).
- Guest order access: capability token (raw shown once, SHA-256 stored) and HMAC-signed `/o/` slugs (JWT_SECRET-keyed); sequential order ids grant nothing.
- Upload hardening: MIME magic-byte sniffing, 8 MB cap, decompression-bomb guard, validate-then-write.
- Security headers on every response: CSP (hand-built inventory; `unsafe-eval` dev-only), nosniff, SAMEORIGIN, Referrer-Policy, Permissions-Policy; HSTS in production builds only.
- **Weaknesses to be aware of** (code-verified): WhatsApp service API-key comparison is plain string equality; `/status` and `/health` unauthenticated (no secrets returned); `/test*` dev endpoints on the service are unauthenticated (localhost-bound); the middleware admin-page gate falls back to unsigned `jwt.decode` if `JWT_SECRET` is absent from the edge runtime (dev case; ensure `JWT_SECRET` reaches the edge in production); no rate limit on the password-reset *reset* step.

10.4 Database security / RLS

- No Row-Level Security is used. Isolation is by application role checks only; the DB user is a least-privilege app role (some legacy tables are owned by the migration role — hence the FK-less companion tables). All access flows through Prisma with parameterised queries (no raw SQL string building found in routes).

10.5 WhatsApp key & session handling

- `WHATSAPP_API_KEY` (app) must equal `API_KEY` (service); sent as `X-API-Key`/Bearer server-to-server; never exposed to browsers (all staff calls are proxied by Next routes).
- `whatsapp-service/auth/` holds the live WhatsApp Web identity. Never commit, copy, zip-and-share, or restore it onto another machine casually — whoever holds it can send messages as the linked number. `/logout` wipes it deliberately.

10.6 Backup & restoration process

- ADMIN-only ZIP downloads: orders (JSON, incl. security hashes) and products (JSON + full `public/uploads` + manifest/missing-image report).
- Restore is a **non-destructive merge** with `?dryRun=1` validation, chunked upload for large archives, and no-overwrite image writing. It never updates or deletes existing rows; conflicts are reported. It is not transactional and writes no audit rows — run it during a quiet period and review the JSON report.

- Independent database-level backups exist locally (`db-backups/*.dump` , git-ignored) — keep a current dump routine; the in-app backup covers orders/products/images, not users/settings/taxonomy.

10.7 Operational precautions

- Never run two WhatsApp service instances against the same `auth/` dir.
- Rebuild committed Tailwind CSS after class changes; bump `?v=` query strings when changing `/assets` files without changing URLs (cache headers allow 1-day staleness).
- `Setting` row `whatsapp_test_mode` is ON unless explicitly `"false"` — check it before assuming real delivery.
- HTML pages are served `no-cache` -revalidate by design; don't "fix" perceived slowness by caching HTML.

---

11. Known Limitations and Outstanding Work

Verified from the code (file references in earlier sections):

1. **Dead dependencies:** `exceljs` , `recharts` and `jsdom` are declared but imported nowhere in the project's own code. Candidate cleanups (or future features).
2. **No tests, no linting, no CI.** `npm run typecheck` is the only automated gate.
3. **Storefront page sprawl:** 47 HTML files ship in `public/` ; roughly 15 are superseded variants (cart2–7, checkout2–4, home, order-send-modified2/3, houseware2, test pages) plus multiple `.bak*` files, all publicly fetchable. The live chain is: `home5.html → cart.html → checkout5.html → order-send-modified4.html` ; `order-send-modified4.html` still has `<title>Shopping Cart</title>` . Old pages link `checkout2.html` (dead chain).
4. **No Prisma postinstall hook:** `prisma generate` must be run manually after every `npm install` and on any deployment host before/at build time (`DATABASE_SETUP.md` claims it is "already handled by the current build", but `package.json` contains no such hook — verify on the actual build platform).
5. `i18n-data.js` **regeneration depends on an external xlsx pipeline** (`translation-system/apply.cjs` , referenced in the `i18n.js` header) whose source file could not be located in the repo — translations are effectively hand-maintained now.
6. **INQUIRY purchase mode** exists in schema/admin but has no customer-facing behaviour.
7. **Product "versioning" is the audit log only**; duplicate detection is slug-only; no bulk edit; no Excel import/export.
8. **Restore tools are not transactional** and write no audit rows; archived `SEEN` statuses are not in the restore route's valid-status list (they fall back to PENDING on history rows).
9. **Per-process in-memory state** (rate limits, catalog cache, service idempotency map) — single-instance deployment only.
10. **Middleware unsigned-JWT fallback** for page gating when `JWT_SECRET` is absent at the edge — ensure production env propagation.
11. **Cart POST sets (not increments) quantity**; `/api/notifications` PATCH silently no-ops on malformed bodies; profile password change does not revoke the user's *other* sessions (password reset does).
12. **No PWA manifest, robots.txt or sitemap**; admin product list/report endpoints return unpaginated full datasets (fine at current scale, watch at larger scale).
13. **Deployment not configured in-repo** (no vercel.json/Dockerfile/CI). Baileys + local uploads constrain hosting choices (§9).
14. **Baileys is an unofficial library at a release candidate** (`7.0.0-rc14`) — protocol breakage risk; use a dedicated number; in-memory idempotency keys are lost on service restart (order-row claim still protects).
15. **Hardcoded LAN IP** `192.168.1.172` in the `dev:https:certs` mkcert script — update for a different LAN.
16. `whatsapp-service/.env.example` **is referenced by its README but missing** — recreate it when onboarding (§7.2 lists the contents).
17. **Housekeeping:** root contains one-off scripts, dumps, screenshots and backup folders (mostly git-ignored). Safe to archive, but do not delete `DATABASE_SETUP.md` , `docs/SECURITY-NOTES.md` , or anything under `scripts/` without review (`wa-service-manager.mjs` is load-bearing for `next dev`).
18. **Git state at handover:** working tree has uncommitted changes — modified `.gitignore` , untracked `instrumentation.js` , `scripts/wa-service-manager.mjs` , and the whole `whatsapp-service/` directory (i.e. the latest WhatsApp automation work is **not committed** to any remote). Commit and push before archiving the machine.

---

12. Developer Quick Reference

Item	Value
Project root	`V:\14 SEPT WITH ONLY ORDER\5-8-26\backup 4-8-26\nassim-main`
Node / npm	v24.18.1 / 11.16.0 (min Node 20)
Framework	Next.js **15.5.24** (App Router), React 19.2.7 (admin only), TypeScript 5.9.3
DB	Neon PostgreSQL 18, Prisma 6.19.3, 18 migrations
Auth	DB-backed sessions, HS256 JWT cookie `nassim_sid`, bcrypt-12, roles CUSTOMER/STAFF/ADMIN(+dormant SUPERADMIN)
WhatsApp	Baileys 7.0.0-rc14 standalone Express service, 127.0.0.1:4000, API-key auth
Email / Push	Resend (password reset), Web Push VAPID (staff new-order)
Local URLs	App `http://localhost:3000` · Admin `/admin/login` · Staff `/internal-orders2.html` (legacy page: `/internal-orders.html`) · WA service `http://127.0.0.1:4000/health`
Key env vars (names)	`DATABASE_URL`, `DIRECT_URL`, `JWT_SECRET`, `SESSION_COOKIE_NAME`, `ADMIN_SEED_*`, `RESEND_API_KEY`, `EMAIL_FROM`, `APP_URL`, `APP_BASE_URL`, `VAPID_*`, `WHATSAPP_SERVICE_URL`, `WHATSAPP_API_KEY`, `WHATSAPP_TEST_RECIPIENT`, `WA_SERVICE_AUTO_START`; service: `PORT`, `HOST`, `API_KEY`, `AUTH_DIR`
Install	`npm install` (root) + `npm install` (in `whatsapp-service/`)
DB setup	`npx prisma generate` → `npx prisma migrate deploy` → `npm run seed`
Start dev (both services)	**`npx next dev`** (stops both with Ctrl+C)
WA service manual	`cd whatsapp-service && npm run build && npm start`
Build / start prod	`npm run typecheck` → `npm run build` → `npm run start`
CSS rebuild	`npm run build:css` (storefront) · `npm run build:css:admin` (admin)
API code	`app/api/**/route.ts` (49 routes)
Core libs to read first	`lib/auth.ts`, `lib/order-workflow.ts`, `lib/whatsapp-automation.ts`, `middleware.ts`, `instrumentation.js`, `next.config.mjs`, `prisma/schema.prisma`, `DATABASE_SETUP.md`
Frontend entry points	`public/home5.html`, `public/checkout5.html` + `public/assets/js/checkout.js`, `public/internal-orders2.html`, `app/admin/AdminApp.tsx`
Storage keys	localStorage `nassim_cart`, `nassim_wishlist`, `nassim-lang`, `color-theme`; sessionStorage `nassim_checkout_draft`
Never touch	`whatsapp-service/auth/` (live session), `.env` values, `certs/`, `db-backups/`

End of document. Verification note: every technology, version, endpoint, model and behaviour above was read from the repository's source on 10 November 2026. Items that could not be verified from the code are flagged inline (translation xlsx pipeline location, live deployment status, presence of `.env.example` for the WhatsApp service).